Envío de Evidencias MVP: PichangaBook Arequipa

Avance de Backend, Frontend, Diagramas y Planificación

Formación de Empresas de Base Tecnológica I — Universidad Católica San Pablo

Rodrigo Sierra Samuel Iman

5 de noviembre de 2026

1 Resumen del Avance del MVP

En este documento presentamos las evidencias de avance del Producto Mínimo Viable (MVP) de **PichangaBook Arequipa** (repositorio `github.com/RodrigoSierr/PichangaBook`). El proyecto resuelve tres problemas al alquilar canchas sintéticas en Arequipa: las reservas duplicadas entre recepción y WhatsApp, la dificultad de cobrar la cuota a cada amigo del partido y la falta de suplentes de último minuto. Hasta la fecha hemos implementado el esquema de base de datos en **PostgreSQL**, el servidor backend en **NestJS con Socket.io** y las interfaces de usuario en **HTML5/Tailwind CSS** (panel web de dueños) y **Flutter/Dart** (aplicación móvil para jugadores).

2 Diagramas del Sistema

2.1 Diagrama de Arquitectura Backend–Frontend

El siguiente diagrama muestra cómo se comunican el panel web de recepción y la aplicación móvil con los módulos del backend en NestJS y la base de datos PostgreSQL mediante peticiones REST y eventos en tiempo real por WebSockets:

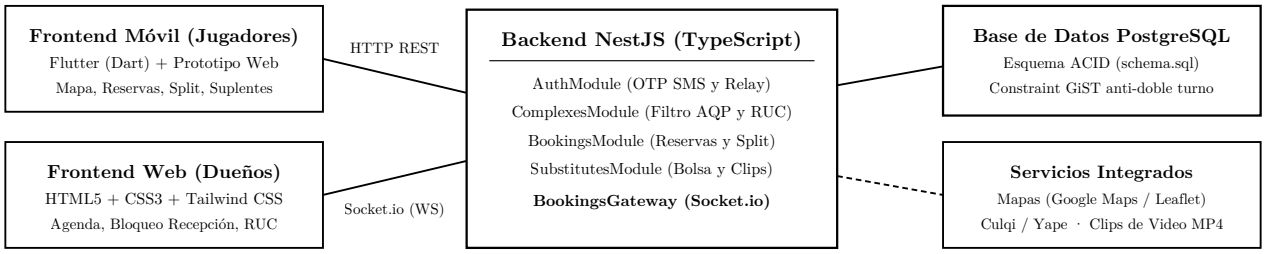


Figura 1: Diagrama de arquitectura general entre el Frontend (Web/Móvil), el Backend NestJS y PostgreSQL.

2.2 Diagrama Entidad-Relación de la Base de Datos

El modelo relacional implementado en `backend-nestjs/schema.sql` se compone de las siguientes entidades principales y sus relaciones de integridad referencial:

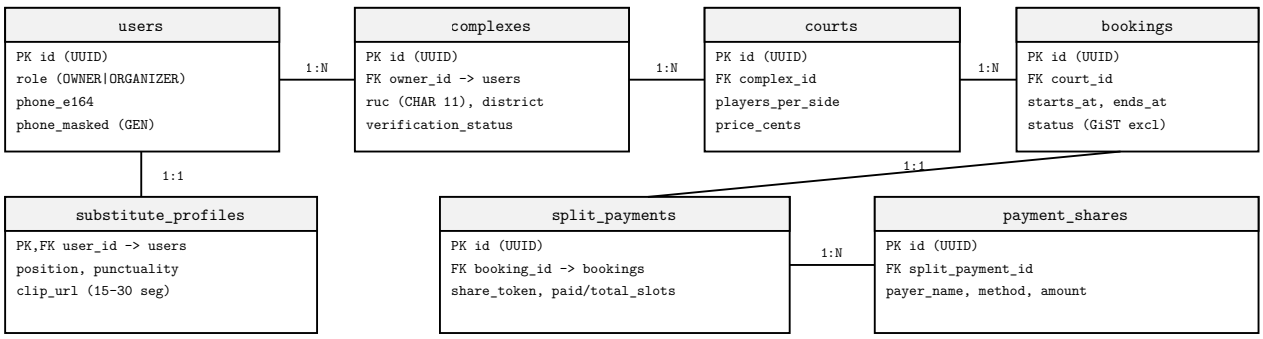


Figura 2: Diagrama Entidad-Relación de las tablas principales en PostgreSQL.

3 Evidencias de Código (Backend, Base de Datos y Frontend)

3.1 Esquema en PostgreSQL y restricción anti-doble reserva (`schema.sql`)

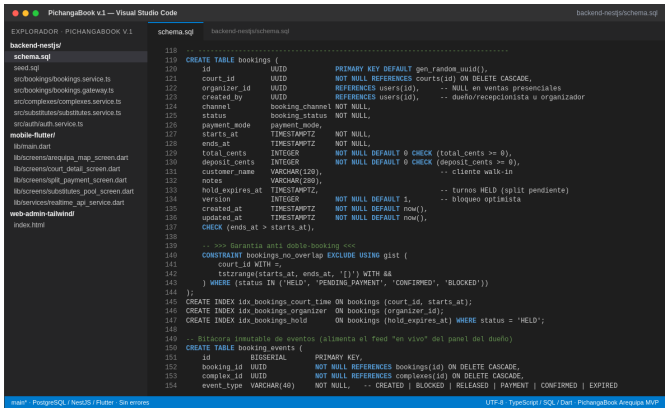


Figura 3: Código SQL de la tabla `bookings` y su restricción GiST en `backend-nestjs/schema.sql`.

En la captura se observa el archivo `backend-nestjs/schema.sql` con la definición de la tabla `bookings` y la tabla de auditoría `booking_events`.

Cada reserva guarda la cancha (`court_id`), el canal de origen (`APP`, `RECEPTION` o `PHONE`), el estado del turno y el rango horario (`starts_at` y `ends_at`). En las líneas 140 a 143 programamos la restricción `CONSTRAINT bookings_no_overlap EXCLUDE USING gist`, que impide a nivel de base de datos que dos reservas activas ocupen la misma cancha en el mismo horario.

3.2 Lógica de reservas y pagos divididos en NestJS (`bookings.service.ts`)

El servicio `BookingsService` (en `src/bookings/bookings.service.ts`) gestiona la creación de reservas, el bloqueo rápido desde recepción y el avance de los pagos divididos.

Como muestra la captura del código en las líneas 94 a 126, cuando un organizador elige la modalidad `SPLIT`, el backend abre una transacción SQL, inserta el registro en `split_payments` calculando el monto individual por jugador (`amount_per_slot_cents`), registra el primer abono en `payment_shares` y guarda el evento en `booking_events` para notificar al panel del dueño.

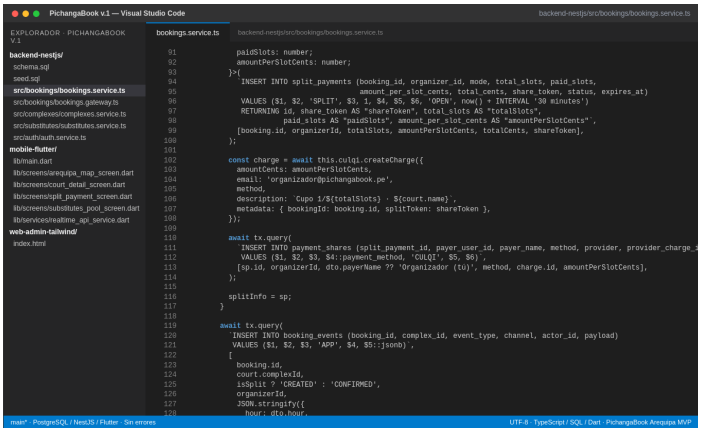


Figura 4: Transacción de reserva y creación de pago dividido en `bookings.service.ts`.

3.3 Gateway de sincronización en tiempo real con Socket.io (`bookings.gateway.ts`)

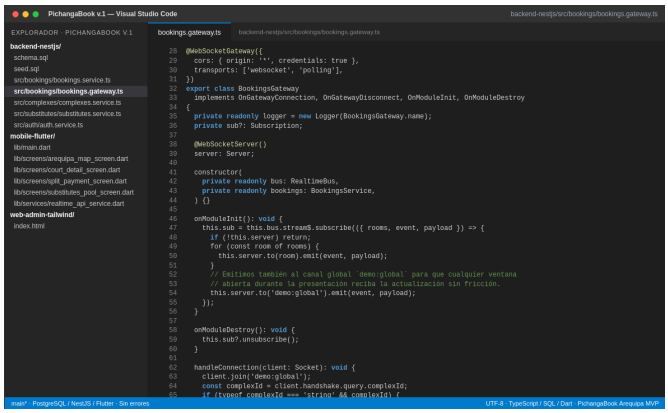


Figura 5: Implementación de `BookingsGateway` con Socket.io en NestJS.

Para que los bloqueos hechos en recepción y los abonos del pago dividido se reflejen sin recargar la página, implementamos la clase `BookingsGateway` en `src/bookings/bookings.gateway.ts` usando `@nestjs/websockets` y **Socket.io**.

En las líneas 46 a 65 se aprecia cómo el gateway escucha el bus interno de eventos (`RealtimeBus`) y emite las actualizaciones (`slot:updated` y `split:updated`) hacia la sala del complejo deportivo (`complex:<id>`) cada vez que cambia un turno.

3.4 Código del Frontend móvil en Flutter (`arequipa_map_screen.dart`)

En la carpeta `/mobile-flutter/lib` estructuramos las vistas móviles en **Dart / Flutter** conectadas al backend mediante el servicio `RealtimeApiService`.

La captura muestra el archivo `arequipa_map_screen.dart`, donde definimos el widget con estado `ArequipaMapScreen`, la posición inicial de la cámara sobre las coordenadas de Arequipa (`LatLng(-16.3880, -71.5430)`) y la lista de distritos (Cayma, Yanahuara, Cerro Colorado, J.L. Bustamante y Sachaca) que filtra las canchas consultando la API del backend.

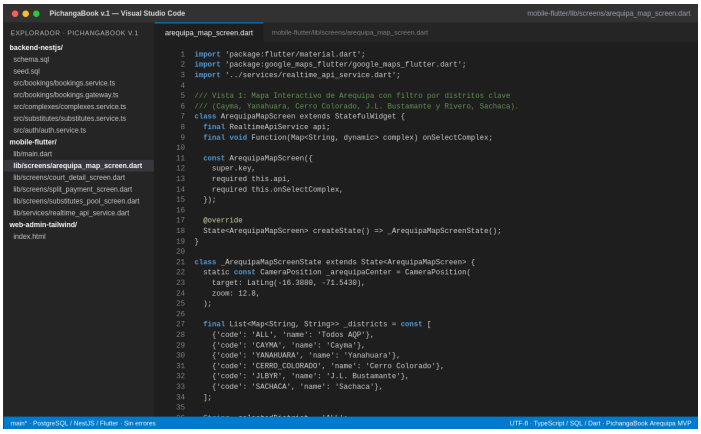


Figura 6: Código en Flutter (Dart) de la pantalla del mapa de Arequipa.

4 Evidencias Visuales del Frontend Conectado al Backend

4.1 Panel web para dueños de complejos deportivos

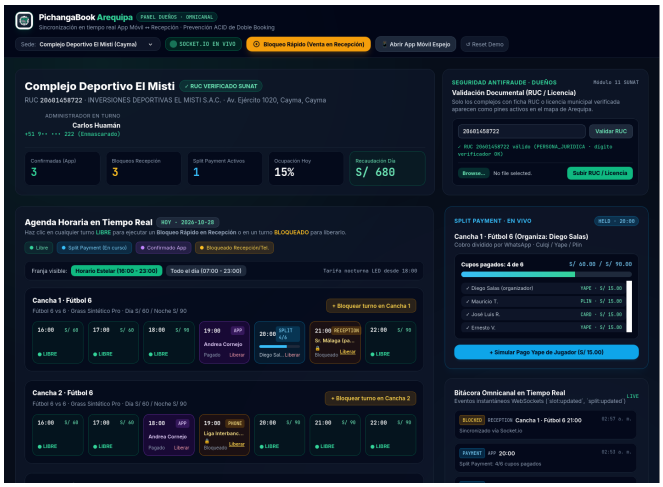


Figura 7: Panel web de administración para dueños de complejos corriendo en el navegador.

El panel web (`/web-admin-tailwind/index.html`) fue maquetado con **HTML5** y **Tailwind CSS**. Al abrirse se conecta al backend NestJS mediante `GET /api/complexes` y al canal de Socket.io.

Muestra los indicadores del complejo deportivo, el validador de RUC con el algoritmo módulo 11 de SUNAT, la parrilla horaria en vivo de cada cancha y el monitor lateral de pagos divididos junto con la bitácora de eventos recibidos por WebSockets.

4.2 Bloqueo rápido de turnos en recepción

Cuando un cliente paga en efectivo en la boletería o reserva por teléfono, el encargado abre este modal en el panel web, elige la cancha, la hora y el nombre del grupo, y pulsa *"Bloquear Turno Ahora"*.

La interfaz envía una petición `POST /api/bookings/reception-block` al backend NestJS, el cual guarda el turno con estado `BLOCKED` en PostgreSQL y emite el evento por Socket.io para que en la aplicación móvil aparezca ocupado al instante.

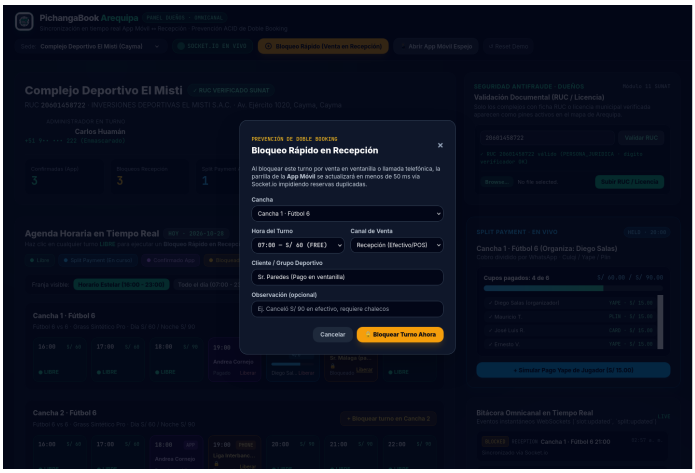


Figura 8: Modal de bloqueo rápido en recepción para ventas presenciales o telefónicas.

4.3 Mapa de complejos y filtros por distritos de Arequipa



Figura 9: Vista móvil del mapa de Arequipa y complejos filtrados por distrito.

La pantalla principal de la app móvil muestra el mapa de Arequipa con pines que indican el precio por hora de cada complejo deportivo y botones superiores para filtrar por distrito (Cayma, Cerro Colorado, Yanahuara y José Luis Bustamante y Rivero).

Al seleccionar un distrito, el frontend consulta `GET /api/complexes?district=...` en el backend NestJS, el cual devuelve únicamente los complejos con RUC verificado en PostgreSQL ordenados por cercanía en kilómetros usando la fórmula de Haversine.

4.4 Detalle de cancha y parrilla de disponibilidad en vivo

Al elegir un complejo se muestra su fotografía, calificación, pestañas para cambiar entre sus canchas (Cancha 1 Fútbol 6, Cancha 2 Fútbol 6, Cancha 3 Fútbol 8) y las reseñas escritas por otros jugadores.

La parrilla horaria consulta los turnos del día en el backend y se mantiene suscrita por WebSockets: muestra en verde los turnos disponibles, en morado los pagados por app, en azul los que están recolectando pagos divididos (`4/6`) y en ámbar los bloqueados en recepción.

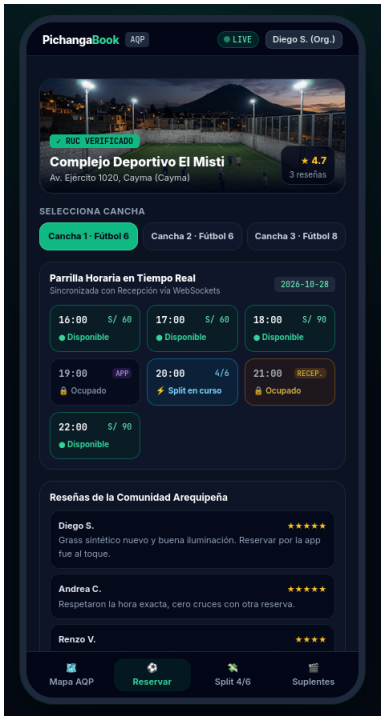


Figura 10: Parrilla horaria sincronizada y reseñas del complejo deportivo.

4.5 Pantalla de pagos divididos (Split Payment)



Figura 11: Contador de progreso de pago dividido (4 de 6 cupos pagados).

En esta vista el organizador genera un link de cobro fraccionado para compartirlo con su grupo por WhatsApp y sigue en vivo cuántos compañeros ya abonaron su parte (por ejemplo, *"Cupos pagados: 4 de 6 — S/ 60.00 / S/ 90.00"*).

Cada vez que un jugador abona su cuota de S/ 15.00 eligiendo Yape, Plin o tarjeta, el frontend invoca `POST /api/bookings/split/:token/pay` en el backend NestJS. Al completarse el sexto cupo, la reserva pasa automáticamente de `HELD` a `CONFIRMED`.

4.6 Bolsa de suplentes verificados con clips de video

La pestaña de bolsa de suplentes permite convocar jugadores de emergencia filtrando por posición en la cancha (`Arquero`, `Defensa`, `Mediocampo` y `Delantero`) mediante el endpoint `GET /api/substitutes`.

Cada tarjeta incluye un reproductor de video con un clip corto de 15 a 30 segundos para comprobar el nivel técnico del jugador antes de invitarlo, además de mostrar su distrito, su porcentaje de puntualidad, partidos jugados y el botón de convocatoria segura.



Figura 12: Tarjetas de suplentes verificados con reproductor de video de 15 a 30 s.

4.7 Seguridad: Verificación por SMS (OTP) y chat con número oculto

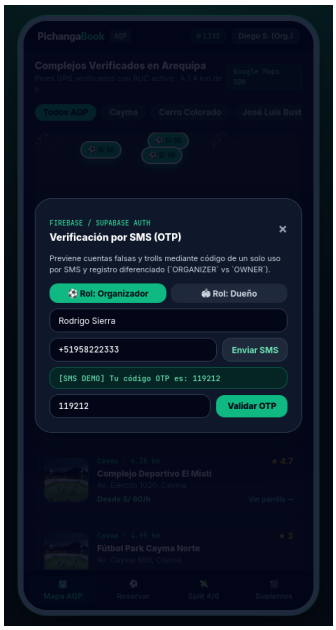


Figura 13: Registro por rol y validación por SMS (OTP).

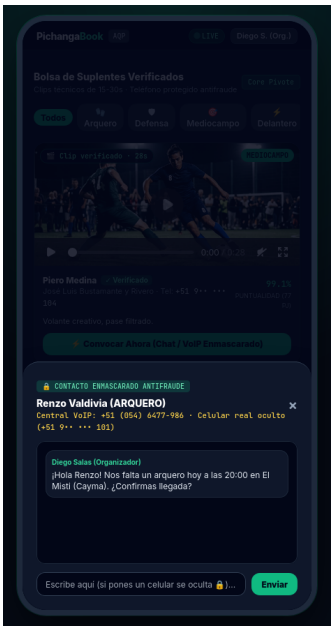


Figura 14: Chat interno con número celular enmascarado.

En el módulo de autenticación implementamos el registro diferenciado por rol (`Organizador` vs. `Dueño`) validado mediante un código temporal de 6 dígitos por SMS (`POST /api/auth/otp/send` y `/verify`) para evitar cuentas falsas. Además, al convocar a un suplente se abre un canal interno de contacto (Figura 14) donde el teléfono personal se muestra siempre enmascarado (`+51 9•• ••• 101`) y el backend oculta cualquier número celular escrito en los mensajes.

5 Avance para la Semana Siguiente (6 al 12 de noviembre de 2026)

Para la entrega de la próxima semana nos enfocaremos en reemplazar los adaptadores simulados por integraciones en modo de prueba real y cerrar los flujos pendientes entre el backend y la aplicación móvil:

Área	**Tarea programada**	**Entregable / Evidencia para la próxima semana**
Backend (Pagos)	Integración con el entorno Sandbox real de Culqi / Yape.	Conexión de `CulqiService` con llaves de prueba reales para procesar abonos del Split Payment y recibir webhooks firmados.
Backend y BD	Liberación automática de reservas vencidas (Cron Job).	Tarea programada en NestJS que cambie a `EXPIRED` los turnos en estado `HELD` si no completan las 6 cuotas antes de `hold_expires_at`.
Backend y Cloud	Subida de clips cortos de suplentes a Cloudinary.	Endpoint para que un jugador suplente suba su propio video de 15 a 30 segundos validando duración y peso del archivo.
Frontend Web	Historial de ingresos y exportación de reportes para dueños.	Vista adicional en el panel web de administración con el resumen semanal de reservas por cancha y descarga en formato CSV.
Frontend Móvil	Compilación de APK en Flutter y pruebas en celulares Android.	Generación del archivo instalable APK conectado al backend desplegado en la nube (Render) y pruebas con usuarios locales.

Tabla 1: Cronograma de tareas y entregables comprometidos para la semana del 6 al 12 de noviembre de 2026.